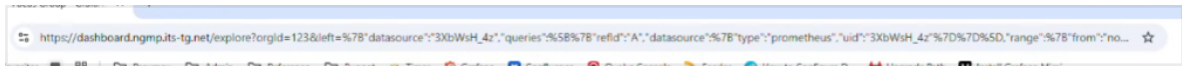
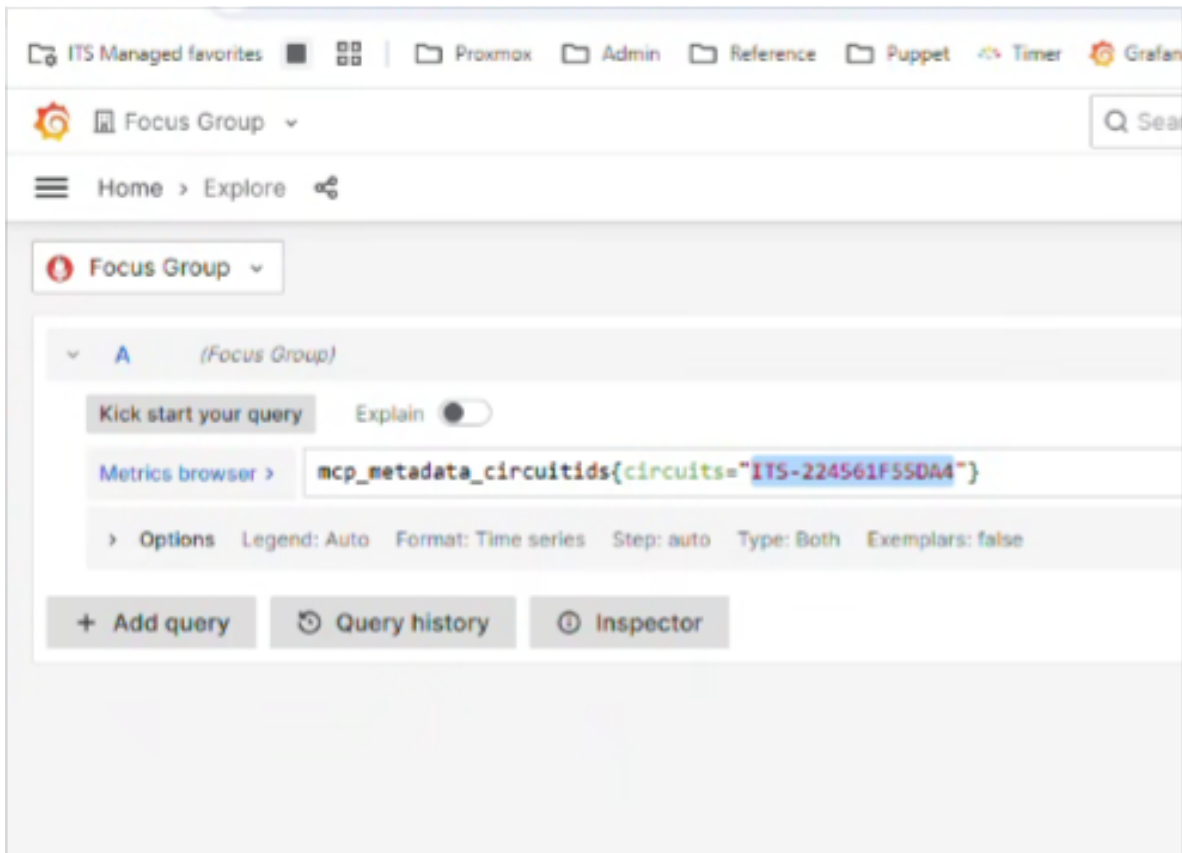


ITS Api Call to Prometheus

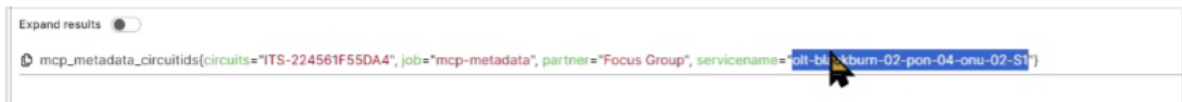
Actually calling Mimir, a multitenant version of prometheus



Header contains UUID specific to client, their should be in SF
Find service name for circuit using circuit ref. (How do we know the right circuit- client will have picked it from portal)



Gives circuit name back:



Get status using circuit name

Gives boolean back to say it's up:



Can look at signal strength too

Can do aggregation queries and timeframe queries too to see if it's back up - not in scope

Could get a null response because system isn't/can't be monitored, if so escalate

Email discussion:

Hi Frank,

I've just come across a very important caveat for the line checks. I've made a very brief note in the spreadsheet, but we should probably pass this along to UC explicitly...

Prometheus has two types of queries, instant and range. Instant queries are the sort of thing we'll be doing for a line check, as they essentially ask what the last recorded state of play was. So for example ONU status is available at:

```
statapp_onu_status_status{instance="olt-chorley-01-xgspon-01-onu-01-S1"}.
```

This query will, in an ideal world return the very latest data on the status of that device. The problem is however, Prometheus goes to great pains to keep data fresh. If there is a fault on the device that prevents polling completely, then the above instant query will eventually become stale, and Prometheus will return no data. That could present the appearance that we have no record of it, which is clearly a bad thing. There is a solution though...

```
last_over_time(statapp_onu_status_status{job="statapp"}[24h])
```

The `last_over_time` function instructs Prometheus to ignore the freshness

checks it normally conducts, and always give the last datapoint recorded. The 24h (in bold) specifies how far you want to "look back" and can be specified as 5m, 1d, 48h etc. For safety, I'd suggest we always wrap line test queries in this function.

https://prometheus.io/docs/prometheus/latest/querying/functions/#aggregation_over_time

thanks,
Iain

From: Iain Smith <iain.smith@itstechnologygroup.com>
Sent: Tuesday, October 8, 2024 2:03 PM
To: Frank Collins <frank.collins@itstechnologygroup.com>
Subject: Re: Prometheus access for UC - IP Addresses

Hi,

For UC, as promised...

Firstly, all queries must contain an X-Scope-OrgID header. In this case, you can use 5b4edc75-bd6f-4e4c-9c87-591c78d0ac40 for Triangle Networks. Omit that, and you'll get empty query results.

Here are some very basic examples for Mimir queries:

GET https://mimir.its-tg.net/api/v1/query?query=cpe_metadata_circuitids

```
"metric": {  
  "__name__": "cpe_metadata_circuitids",  
  "circuits": "ITS-23352F22DB99",  
  "compliant": "1",  
  "instance": "nicheframes-bs67ae.cpe.its-tg.net",  
  "job": "metadata",  
  "partner": "Triangle Networks Limited"  
},  
"value": [  
  1728391617,  
  "0"  
]
```

This returns a list of circuit IDs, which are mapped to "instances" (being a reference to the hostname of the CPE in question). The process should be to look up the instance name on the cpe_metadata_circuitids metric, and the results can be used to query other metrics.

For example:

GET [https://mimir.its-tg.net/api/v1/query_range?query=up{instance=\"nicheframes-bs67ae.cpe.its-tg.net\"}&start=2024-10-07T00:00:00.000Z&end=2024-10-08T00:00:00.000Z&step=300s](https://mimir.its-tg.net/api/v1/query_range?query=up{instance=\)

```
"metric": {
  "__name__": "up",
  "instance": "nicheframes-bs67ae.cpe.its-tg.net",
  "ipaddress": "100.96.74.75",
  "job": "blackbox-ping",
  "partner": "Triangle Networks Limited"
},
"values": [
  [
    1728259200,
    "1"
  ],
  [
    1728259500,
    "1"
  ],
]
```

"Up" in this case is simple ping test, with the result being an array of UNIX timestamps and (in this case) booleans. Start, end and step should be fairly self-explanatory.

I've highlighted the PromQL query in bold, as that's the key part. Afraid I'll have to refer you to the documentation regarding the basics of how to make queries: <https://prometheus.io/docs/prometheus/latest/querying/basics/>. In many cases though, it will be perfectly sufficient to cut and paste the queries from the Grafana dashboards.

thanks,
Iain

From: Iain Smith <iain.smith@itstechnologygroup.com>
Sent: Tuesday, October 8, 2024 9:24 AM
To: Frank Collins <frank.collins@itstechnologygroup.com>
Subject: Re: Prometheus access for UC - IP Addresses

Hi,

I've added the IPs. If they want to do a quick connectivity check they can do so with this: <https://mimir.its-tg.net/api/v1/status/buildinfo>.

Thanks for chasing that.

thanks,
Iain